

UNIVERSIDAD NACIONAL DE CÓRDOBA

Facultad de Ciencias Exactas, Físicas y Naturales



Trabajo Práctico Final

Red de Petri / Sistema de Agencia de Viajes

Programación Concurrente

Profesores:

VENTRE, LUIS ORLANDO

LUDEMANN, MAURICIO

Grupo: *Deadlock Dodgers*

PALACIO, LUIS ENRIQUE (40.609.862)

SAILLEN, SIMON (42.978.650)

TRACHTA, AGUSTÍN (43.271.890)

VARGAS, RODRIGO SEBASTIÁN (42.016.802)

ZUÑIGA, IVAN (43.552.893)

2024

Índice

1. DESARROLLO	2
1.1. PROPIEDADES DE LA RED	2
1.1.1. Análisis Teórico	2
1.1.2. Análisis por Software	4
2. IMPLEMENTACIÓN	5
2.1. Tabla de Estados del Sistema	5
2.2. Tabla de Eventos del Sistema	6
2.3. Responsabilidad de hilos	7
2.3.1. Aplicación del algoritmo 4.1: máximo de hilos activos	7
2.3.2. Aplicación del algoritmo 4.2: segmentación de responsabilidades	9
2.3.3. Aplicación del algoritmo 4.3: capacidad por segmento	11
2.3.4. Workers implementados	11
2.3.5. Relación entre el modelo y la implementación	12
3. MONITOR DE CONCURRENCIA	13
3.1. Estructura del Monitor	13
3.2. Mecanismo de Señalización: Signal and Exit	14
3.3. Diagrama de Secuencia del Monitor	14
3.4. Diagrama de Clases	14
3.5. Flujo de Disparo de una Transición	14
3.6. Independencia de la Lógica	15
4. TIEMPO	16
4.1. Análisis Teórico	16
4.2. Análisis Práctico	16
5. POLÍTICAS	17
5.1. Objetivo y alcance de Policy	17
5.2. Conflictos estructurales y oportunidades reales	18
5.3. Integración con el monitor	18
5.4. Selección proporcional mediante residuo	19
5.5. Modos NONE, BALANCED y PRIORITIZED	20
5.6. Resultados observados	20
5.7. Alcance y limitaciones	22
5.8. Conclusión	23
6. ANÁLISIS DE T-INVARIANTES CON REGEX	23
6.1. Resultados Regex	24
7. CONCLUSIONES	24

1. DESARROLLO

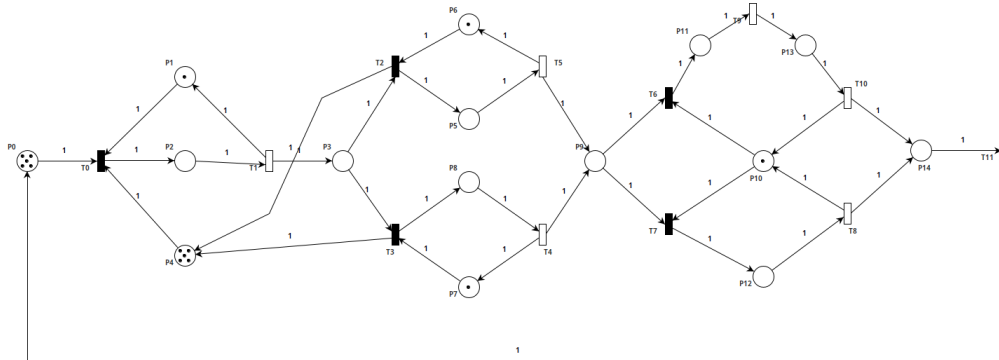


Figura 1: Red de Petri - Sistema de Agencia de Viajes

Referencias:

P1, P4, P6, P7, P10	Recursos compartidos del sistema
P0	Buffer de entrada de clientes
P2	Ingreso a la agencia
P3	Sala de espera
P5, P8	Gestión de reservas
P9	Buffer aceptación/rechazo de reserva
P12	Cancelación de reserva
P11	Confirmación de reserva
P13	Pago de reserva
P14	Instancia previa al retiro del cliente

1.1. PROPIEDADES DE LA RED

1.1.1. Análisis Teórico

Seguridad y acotamiento

Una red es segura si ninguna plaza puede contener más de un token en los marcados alcanzables. La red analizada no es segura. Las cotas máximas alcanzables son 5 tokens para P_0 , P_3 , P_4 , P_9 y P_{14} , y un token para cada una de las demás plazas. Por lo tanto, la red es acotada, pero no segura.

Conservación mediante P-invariantes

Los P-invariantes expresan sumas de tokens que se mantienen constantes en cada disparo. Entre ellos se encuentran

$$\begin{aligned}
M(P_1) + M(P_2) &= 1, & M(P_5) + M(P_6) &= 1, \\
M(P_7) + M(P_8) &= 1, & M(P_{10}) + M(P_{11}) + M(P_{12}) + M(P_{13}) &= 1, \\
M(P_2) + M(P_3) + M(P_4) &= 5,
\end{aligned}$$

y el invariante global de población

$$\begin{aligned}
M(P_0) + M(P_2) + M(P_3) + M(P_5) + M(P_8) + M(P_9) \\
+ M(P_{11}) + M(P_{12}) + M(P_{13}) + M(P_{14}) = 5.
\end{aligned} \tag{1}$$

Estos invariantes justifican la conservación de los cinco clientes, de los recursos unitarios y de la capacidad de la etapa de ingreso. También permiten obtener cotas para las plazas. Por sí solos, no demuestran vivacidad ni ausencia de deadlock.

Como los P-invariantes positivos cubren todas las plazas, la red es conservadora: existe una suma ponderada de tokens que permanece constante durante los disparos.

Vivacidad y ausencia de deadlock

Estas propiedades deben analizarse sobre los marcados alcanzables y no deducirse únicamente de los P-invariantes o de la existencia de ciclos. Para el marcado inicial y las reglas estructurales de la red, el análisis de alcanzabilidad obtuvo 618 marcados, sin encontrar un marcado muerto. Además, desde cada marcado alcanzable es posible regresar al marcado inicial, por lo que M_0 es un estado hogar y la red es reversible respecto de M_0 . Como desde M_0 es posible alcanzar cada transición, desde cualquier marcado todas las transiciones pueden volver a habilitarse. Bajo este análisis, la red P/T es viva y no presenta deadlocks.

El ciclo $P_{14} \xrightarrow{T_{11}} P_0$ permite recircular los clientes, pero no demuestra por sí solo esas propiedades. La conclusión anterior corresponde al modelo estructural. La ejecución Java agrega restricciones temporales, colas, paths fijos de los workers y una política de handoff, por lo que su comportamiento debe verificarse también mediante la ejecución y sus logs.

La red tampoco es persistente. En los conflictos T_2/T_3 y T_6/T_7 , ambas transiciones pueden estar sensibilizadas simultáneamente, pero el disparo de una puede deshabilitar a la otra al consumir una plaza compartida. Esta propiedad explica la necesidad de resolver conflictos durante el handoff.

1.1.2. Análisis por Software

Invariantes de Plaza (P-invariantes)

Petri net state space analysis results

Bounded	true
Safe	false
Deadlock	false

P-Invariants

P0	P1	P2	P3	P4	P5	P6	P7	P8	P9	P10	P11	P12	P13	P14
0	1	1	0	0	0	0	0	0	0	0	0	0	0	0
0	0	1	1	1	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	1	1	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	1	1	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	1	1	1	1	0
1	0	1	1	0	1	0	0	1	1	0	1	1	1	1

The net is covered by positive P-Invariants, therefore it is bounded.

Ecuaciones de Invariantes de Plaza

P-Invariant equations

$$\begin{aligned}
 M(P1) + M(P2) &= 1 \\
 M(P2) + M(P3) + M(P4) &= 5 \\
 M(P5) + M(P6) &= 1 \\
 M(P7) + M(P8) &= 1 \\
 M(P10) + M(P11) + M(P12) + M(P13) &= 1 \\
 M(P0) + M(P2) + M(P3) + M(P5) + M(P8) + M(P9) + M(P11) + M(P12) + M(P13) + M(P14) &= 5
 \end{aligned}$$

Los P-invariantes mostrados expresan la conservación de clientes y recursos a lo largo del proceso. En particular, el invariante global tiene valor 5 y acota la cantidad de clientes distribuidos entre el buffer y las etapas del proceso. Esta evidencia permite afirmar conservación y acotamiento, pero no permite afirmar por sí sola que la red sea viva o que no tenga deadlocks.

Invariantes de Transición (T-invariantes)

T-Invariants

T0	T1	T2	T3	T4	T5	T6	T7	T8	T9	T10	T11
1	1	0	1	1	0	0	1	1	0	0	1
1	1	0	1	1	0	1	0	0	1	1	1
1	1	1	0	0	1	0	1	1	0	0	1
1	1	1	0	0	1	1	0	0	1	1	1

The net is covered by positive T-Invariants, therefore it might be bounded and live.

Los T-invariantes representan secuencias cíclicas cuya ejecución neta conserva el marcado. La numeración se alinea con `regex/TInvariants.yaml`:

- `inv_1`: rama superior y confirmación, pago y retorno.
- `inv_2`: rama superior y cancelación y retorno.
- `inv_3`: rama inferior y confirmación, pago y retorno.
- `inv_4`: rama inferior y cancelación y retorno.

La existencia de estas secuencias muestra que el proceso tiene recorridos repetibles. No demuestra por sí sola que todos los marcados alcanzables tengan una salida, que todos los workers completen un ciclo ni que la ejecución esté libre de deadlocks.

2. IMPLEMENTACIÓN

2.1. Tabla de Estados del Sistema

En una Red de Petri, el estado global del sistema está dado por el marcado completo de sus 15 plazas. La siguiente tabla indica qué representa cada plaza y el valor que toma en el marcado inicial M_0 .

Cuadro 1: Estado inicial de cada plaza

Plaza	Representa	m_0
P_0	Buffer inicial de clientes.	5
P_1	Habilita el ingreso mediante T_0 ; T_1 lo regenera.	1
P_2	Cliente que ingresó a la agencia y espera T_1 .	0
P_3	Sala de espera; punto de decisión entre T_2 y T_3 .	0
P_4	Capacidad de la etapa de ingreso; T_0 consume un token y T_2/T_3 lo regeneran.	5
P_5	Cliente en la rama superior, entre T_2 y T_5 .	0
P_6	Recurso del agente superior; T_2 lo consume y T_5 lo regenera.	1
P_7	Recurso del agente inferior; T_3 lo consume y T_4 lo regenera.	1
P_8	Cliente en la rama inferior, entre T_3 y T_4 .	0
P_9	Cola de decisión de reserva; recibe ambas ramas.	0
P_{10}	Recurso compartido de la decisión final; lo regeneran T_8 y T_{10} .	1
P_{11}	Reserva confirmada, pendiente de T_9 .	0
P_{12}	Reserva cancelada, pendiente de T_8 .	0
P_{13}	Pago en proceso, pendiente de T_{10} .	0
P_{14}	Clientes listos para retornar al buffer mediante T_{11} .	0

El valor de m_0 indica la cantidad de tokens de cada plaza en el marcado inicial; no equivale a la cantidad de disparos inmediatamente sensibilizados. Por ejemplo, T_2 requiere tokens en P_3 y P_6 , y T_3 en P_3 y P_7 . Las plazas de la tabla tampoco representan estados globales mutuamente excluyentes, ya que varias pueden contener tokens simultáneamente en un mismo marcado.

2.2. Tabla de Eventos del Sistema

Cuadro 2: Tabla de Eventos del Sistema

Evento	Transición	Descripción
E_0	T_0	$P_0 + P_1 + P_4 \rightarrow P_2$: el cliente entra a la agencia y consume el recurso de ingreso y un token de capacidad.
E_1	T_1	$P_2 \rightarrow P_1 + P_3$: el cliente pasa a la sala de espera y regenera P_1 .
E_2	T_2	$P_3 + P_6 \rightarrow P_4 + P_5$: inicia la gestión en la rama superior.
E_3	T_3	$P_3 + P_7 \rightarrow P_4 + P_8$: inicia la gestión en la rama inferior.
E_4	T_4	$P_8 \rightarrow P_7 + P_9$: completa la gestión inferior y regenera P_7 .
E_5	T_5	$P_5 \rightarrow P_6 + P_9$: completa la gestión superior y regenera P_6 .
E_6	T_6	$P_9 + P_{10} \rightarrow P_{11}$: confirma la reserva y consume P_{10} .
E_7	T_7	$P_9 + P_{10} \rightarrow P_{12}$: cancela la reserva y consume P_{10} .
E_8	T_8	$P_{12} \rightarrow P_{10} + P_{14}$: completa la cancelación y regenera P_{10} .
E_9	T_9	$P_{11} \rightarrow P_{13}$: pasa de reserva confirmada a pago en proceso.
E_{10}	T_{10}	$P_{13} \rightarrow P_{10} + P_{14}$: completa el pago, regenera P_{10} y deja al cliente listo para retornar.
E_{11}	T_{11}	$P_{14} \rightarrow P_0$: el cliente retorna al buffer de entrada.

Cada fila muestra el efecto completo del disparo: las plazas a la izquierda pierden un token y las de la derecha ganan uno. Así quedan explícitos tanto el avance del cliente como la regeneración de los recursos compartidos.

2.3. Responsabilidad de hilos

La determinación de la cantidad y responsabilidad de los hilos se basa en los algoritmos presentados en las secciones 4.1, 4.2 y 4.3 del paper de Ventre y Micolini sobre sistemas S3PR modelados con Redes de Petri. Estos algoritmos responden preguntas diferentes:

Cuadro 3: Preguntas que responde cada algoritmo del paper.

Algoritmo	Pregunta
4.1	¿Cuál es el máximo de hilos activos simultáneamente según los marcados alcanzables de la red?
4.2	¿Cómo se dividen los T-invariantes en segmentos de responsabilidad cuando existen caminos lineales, forks y convergencias?
4.3	¿Cuál es la capacidad máxima asociada a cada segmento y cuál es la suma de esas capacidades?

La aplicación de estos algoritmos no produce automáticamente la misma cantidad para las tres preguntas. En nuestro caso se obtienen cinco clientes activos como máximo, seis segmentos lógicos y seis workers persistentes. La suma de las capacidades internas de los segmentos es diez bajo la convención de plazas frontera adoptada.

2.3.1. Aplicación del algoritmo 4.1: máximo de hilos activos

El algoritmo 4.1 determina el máximo de actividad que puede expresar el modelo formal. Se aplica en los siguientes pasos.

Paso 1: obtener los T-invariantes

Los cuatro T-invariantes mínimos se expresan como secuencias ordenadas de transiciones. La secuencia es la forma que luego utilizan los workers para recorrer un camino concreto:

$$\begin{aligned}
IT_1 &= \langle T_0, T_1, T_2, T_5, T_6, T_9, T_{10}, T_{11} \rangle, \\
IT_2 &= \langle T_0, T_1, T_2, T_5, T_7, T_8, T_{11} \rangle, \\
IT_3 &= \langle T_0, T_1, T_3, T_4, T_6, T_9, T_{10}, T_{11} \rangle, \\
IT_4 &= \langle T_0, T_1, T_3, T_4, T_7, T_8, T_{11} \rangle.
\end{aligned}$$

Todos los invariantes comparten el ingreso T_0, T_1 y el retorno T_{11} . Las dos elecciones independientes son T_2/T_3 y T_6/T_7 .

Paso 2: obtener las plazas asociadas

Para cada IT_i se construye PL_i , formado por las plazas que son pre o postcondición de alguna transición del invariante. A partir de la matriz de incidencia de esta red se obtienen:

$$\begin{aligned}
PL_1 &= \{P_0, P_1, P_2, P_3, P_4, P_5, P_6, P_9, \\
&\quad P_{10}, P_{11}, P_{13}, P_{14}\}, \\
PL_2 &= \{P_0, P_1, P_2, P_3, P_4, P_5, P_6, P_9, \\
&\quad P_{10}, P_{12}, P_{14}\}, \\
PL_3 &= \{P_0, P_1, P_2, P_3, P_4, P_7, P_8, P_9, \\
&\quad P_{10}, P_{11}, P_{13}, P_{14}\}, \\
PL_4 &= \{P_0, P_1, P_2, P_3, P_4, P_7, P_8, P_9, \\
&\quad P_{10}, P_{12}, P_{14}\}.
\end{aligned}$$

Estos conjuntos se calculan para cada camino. No se incorporan plazas de una rama que no pertenezca al T-invariante analizado.

Paso 3: seleccionar las plazas de acción

El paper indica separar las plazas que representan clientes avanzando por el proceso de aquellas que representan estado idle, recursos o restricciones. Para esta red se tiene:

$$\begin{aligned}
PI_{\text{idle}} &= \{P_0\}, \\
PI_{\text{recursos}} &= \{P_1, P_6, P_7, P_{10}\}, \\
PI_{\text{restricciones}} &= \{P_4\}.
\end{aligned}$$

Por lo tanto, las plazas de acción que se consideran para medir la actividad son:

$$PA = \{P_2, P_3, P_5, P_8, P_9, P_{11}, P_{12}, P_{13}, P_{14}\}. \quad (2)$$

Paso 4: analizar los marcados alcanzables

El algoritmo propone observar cuántos tokens pueden encontrarse en las plazas de acción para los marcados alcanzables de la red. En este caso, el P-invariante global permite obtener ese límite sin enumerar todos los estados:

$$\begin{aligned}
M(P_0) + M(P_2) + M(P_3) + M(P_5) + M(P_8) + M(P_9) \\
+ M(P_{11}) + M(P_{12}) + M(P_{13}) + M(P_{14}) = 5.
\end{aligned} \quad (3)$$

La ecuación indica que la red conserva cinco clientes. Como P_0 es la plaza idle, los clientes que no están en P_0 se encuentran en alguna etapa activa del proceso.

Paso 5: obtener el máximo

La suma de tokens en las plazas de acción es:

$$\sum_{p \in PA} M(p) = 5 - M(P_0) \leq 5. \quad (4)$$

El máximo se alcanza cuando P_0 queda vacío. Por ejemplo, al ejecutar cinco veces el tramo T_0, T_1 , los cinco clientes pueden ubicarse en P_3 :

$$M(P_3) = 5, \quad M(P_0) = 0, \quad (5)$$

En consecuencia, el resultado del algoritmo 4.1 es:

$$H_{\text{activos}}^{\text{máx}} = 5. \quad (6)$$

Este valor es el máximo global de clientes en plazas de acción. No determina la cantidad de objetos **Thread** que debe instanciar la aplicación.

2.3.2. Aplicación del algoritmo 4.2: segmentación de responsabilidades

El algoritmo 4.2 analiza la estructura de los T-invariantes y distingue caminos lineales, forks y convergencias. La responsabilidad se asigna a segmentos con secuencias fijas. Un worker no contiene la decisión sobre qué rama tomar.

Caso 1: camino lineal

Si un T-invariante es lineal y no presenta una decisión compartida con otro camino, sus transiciones consecutivas pueden formar un único segmento:

$$S = [T_a, T_b, T_c]. \quad (7)$$

En nuestra red no existe un T-invariante completamente aislado. Sin embargo, este caso explica por qué cada tramo consecutivo de una rama se mantiene en un mismo worker.

Caso 2: fork en P_3

Los T-invariantes se separan en P_3 entre las alternativas T_2 y T_3 . Según el paper, un fork se divide en un segmento común previo y un segmento para cada rama posterior:

$$\begin{aligned} S_A &= [T_0, T_1], \\ S_B &= [T_2, T_5], \\ S_C &= [T_3, T_4]. \end{aligned}$$

S_A termina antes de la elección. S_B y S_C ejecutan las ramas del agente superior e inferior, respectivamente. Así se evita que un worker tenga que programar una condición del tipo “si ocurre algo, elegir T_2 ; si no, elegir T_3 ”. El monitor y la política resuelven esa elección.

Caso 3: convergencia y nuevo fork en P_9

P_9 recibe tokens de las dos ramas de agentes y, al mismo tiempo, precede el conflicto entre T_6 y T_7 . La convergencia en P_9 es de tipo OR: una ruta produce el token y las rutas no deben sincronizarse mediante una transición que consuma tokens de ambas ramas.

Como no hay una secuencia común entre la llegada a P_9 y el segundo fork, se adopta P_9 como frontera de despacho. Las ramas posteriores quedan:

$$\begin{aligned} S_D &= [T_6, T_9, T_{10}], \\ S_E &= [T_7, T_8]. \end{aligned}$$

Esta asignación aplica el criterio del paper a la topología concreta. La decisión de tratar P_9 como frontera evita asignarla simultáneamente a dos segmentos.

Caso 4: convergencia en P_{14}

El paper indica que después de una convergencia debe existir un único segmento común. En esta red, ambos caminos finales llegan a P_{14} y comparten la transición de retorno:

$$S_F = [T_{11}]. \tag{8}$$

Esto corrige una posible duplicación de T_{11} dentro de las ramas de confirmación y cancelación. T_{11} pertenece al segmento común posterior a la convergencia.

Convención para las plazas frontera

Se adopta la siguiente convención para aplicar los algoritmos a esta red:

- P_3 es la frontera entre S_A y S_B/S_C .
- P_9 es la frontera entre S_B/S_C y S_D/S_E .
- P_{14} pertenece al segmento común S_F .
- P_3 y P_9 continúan formando parte de PA para el máximo global, pero no se asignan como plazas internas de ningún segmento.

El paper define las reglas de separación en forks y convergencias, pero no prescribe una partición única para una plaza que es simultáneamente punto de convergencia y antecedente de otro fork. Por eso esta convención debe declararse como una decisión de aplicación.

2.3.3. Aplicación del algoritmo 4.3: capacidad por segmento

Una vez definidos los segmentos, el algoritmo 4.3 se aplica en cuatro pasos:

1. determinar las plazas de acción internas de cada segmento;
2. proyectar los marcados alcanzables sobre esas plazas;
3. obtener el máximo de tokens para cada segmento;
4. sumar las capacidades máximas individuales.

Con la convención anterior, las plazas internas son:

$$\begin{array}{lll} PA(S_A) = \{P_2\}, & PA(S_B) = \{P_5\}, & PA(S_C) = \{P_8\}, \\ PA(S_D) = \{P_{11}, P_{13}\}, & PA(S_E) = \{P_{12}\}, & PA(S_F) = \{P_{14}\}. \end{array}$$

Para cada segmento se define:

$$h_i = \max_{M \in R(M_0)} \left(\sum_{p \in PA(S_i)} M(p) \right). \quad (9)$$

Las cotas se obtienen de los P-invariantes:

Cuadro 4: Aplicación del algoritmo 4.3 a los segmentos.

Segmento	Plazas internas	h_i	Fundamento
S_A	P_2	1	$M(P_1) + M(P_2) = 1.$
S_B	P_5	1	$M(P_5) + M(P_6) = 1.$
S_C	P_8	1	$M(P_7) + M(P_8) = 1.$
S_D	P_{11}, P_{13}	1	$M(P_{10}) + M(P_{11}) + M(P_{12}) + M(P_{13}) = 1.$
S_E	P_{12}	1	El mismo P-invariante de la etapa de decisión final.
S_F	P_{14}	5	Cota derivada del P-invariante global de población.

La capacidad teórica total por responsabilidades es:

$$H_{\text{segmentos}} = \sum_i h_i = 1 + 1 + 1 + 1 + 1 + 5 = 10. \quad (10)$$

El valor 10 es la suma de máximos calculados para cada segmento por separado. No es el máximo global de clientes activos, porque los máximos de los segmentos no tienen por qué alcanzarse en el mismo marcado.

2.3.4. Workers implementados

El algoritmo 4.2 define los segmentos lógicos. La aplicación los ejecuta con seis workers persistentes, uno por segmento:

Cuadro 5: Correspondencia entre segmentos y workers implementados.

Worker	Segmento	Path	Responsabilidad
Thread-1	S_A	$[T_0, T_1]$	Inicia nuevas vueltas del sistema.
Thread-2	S_B	$[T_2, T_5]$	Ejecuta la rama del agente superior.
Thread-3	S_C	$[T_3, T_4]$	Ejecuta la rama del agente inferior.
Thread-4	S_D	$[T_6, T_9, T_{10}]$	Ejecuta confirmación y pago.
Thread-5	S_E	$[T_7, T_8]$	Ejecuta cancelación.
Thread-6	S_F	$[T_{11}]$	Ejecuta el retorno común y cuenta finalizaciones.

Cada worker recorre repetidamente su path y solicita los disparos al monitor.

2.3.5. Relación entre el modelo y la implementación

Los primeros cinco segmentos están acotados en uno por P-invariantes asociados a recursos de capacidad unitaria. $S_F = [T_{11}]$ es diferente: P_{14} no está vinculada a uno de esos recursos, por lo que su única cota es la población global de la red. En consecuencia, puede contener hasta cinco clientes listos para retirarse.

Aplicado literalmente, el algoritmo 4.3 sugiere:

$$1 + 1 + 1 + 1 + 1 + 5 = 10 \text{ hilos.} \quad (11)$$

La implementación utiliza un worker por segmento:

$$1 + 1 + 1 + 1 + 1 + 1 = 6 \text{ workers.} \quad (12)$$

Para S_F se asigna un único worker porque T_{11} es inmediata y todos los disparos se ejecutan en exclusión mutua dentro del monitor. Agregar más workers no permitiría disparar T_{11} en paralelo y aumentaría la competencia por el mutex. Esta reducción es una decisión de implementación; su efecto sobre el rendimiento debe comprobarse durante la ejecución.

Por lo tanto, 10 es la capacidad teórica obtenida por el algoritmo 4.3, mientras que 6 es la cantidad de workers implementados. El máximo global de clientes activos continúa siendo 5.

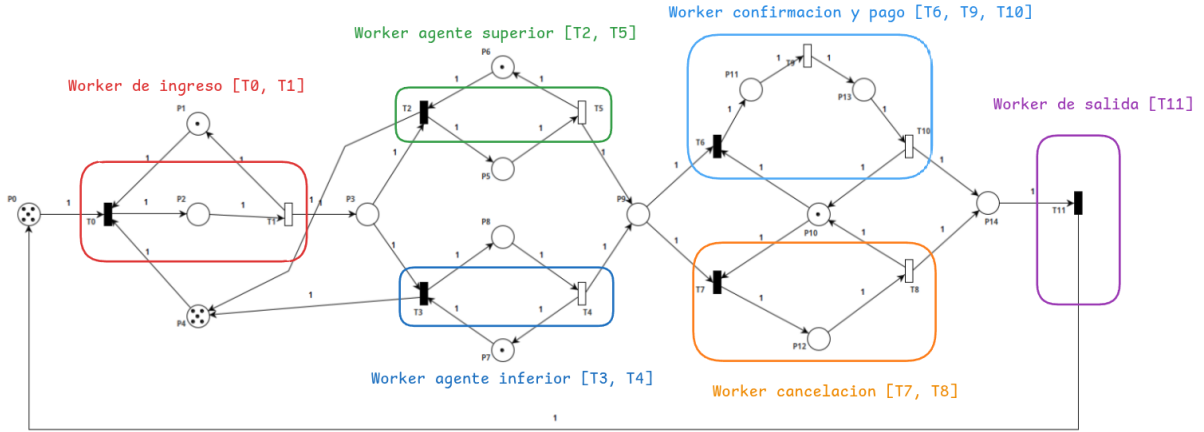


Figura 2: Figura 2: Representación gráfica de la asignación de hilos sobre la Red de Petri.

3. MONITOR DE CONCURRENCIA

El monitor de concurrencia es el núcleo del sistema, responsable de coordinar el acceso de los hilos a la Red de Petri y garantizar que se respeten las reglas de exclusión mutua y sincronización. La implementación se basa en una política de señalización **Signal and Exit (SX)**.

3.1. Estructura del Monitor

El monitor centraliza cuatro responsabilidades fundamentales:

1. **Exclusión Mutua:** Asegurar que solo un hilo a la vez opere sobre el marcado de la RdP.
2. **Gestión de Colas:** Administrar hilos bloqueados que esperan a que una transición se habilite.
3. **Semántica Temporal:** Validar que los disparos ocurran dentro de los rangos de tiempo permitidos.
4. **Registro y consistencia:** Registrar los disparos y comprobar los P-invariantes bajo exclusión mutua.

Para cumplir con esto, la clase **Monitor** utiliza las siguientes estructuras internas:

- **entry:** Un semáforo binario justo, creado con `new Semaphore(1, true)`, que actúa como el mutex de entrada al monitor. Su equidad regula el acceso al monitor, pero no aplica la prioridad de negocio.
- **queues:** Una estructura que mantiene un semáforo, inicializado en 0, y un contador de hilos en espera por cada transición de la red.
- **rdp:** Representación del modelo formal de la Red de Petri, que mantiene el marcado actual, las matrices de incidencia y calcula la sensibilización estructural de las transiciones.
- **policy:** El componente encargado de seleccionar qué hilo bloqueado despertar durante el handoff, entre los candidatos elegibles.

- **time:** Gestor de la semántica temporal.

3.2. Mecanismo de Señalización: Signal and Exit

La implementación del monitor sigue la semántica de señalización conocida como **Signal and Exit**. En este modelo, después de un disparo, si existe un hilo bloqueado cuya transición está sensibilizada, el hilo que realizó el disparo transfiere inmediatamente la posesión lógica del mutex al hilo despertado mediante un *handoff*, sin retener el control del monitor.

Este enfoque se implementa a través de la clase interna **Ownership**, que modela la posesión lógica del semáforo **entry**. El método **handoff()** establece **owned = false** sin invocar **entry.release()**, manteniendo el permiso del semáforo intacto. El hilo despertado, al reanudar su ejecución desde la cola de espera de la transición, invoca **wakeFromQueue()** que establece **owned = true**, asumiendo así la posesión del mismo permiso sin necesidad de competir por él.

Este enfoque tiene dos implicancias críticas en el diseño de nuestro sistema:

- **Transferencia Directa del Mutex:** El hilo despertado no necesita re-adquirir el semáforo **entry**. La posesión se transfiere a través del mecanismo de *handoff*, evitando que compita por el mutex con otros hilos que intenten ingresar al monitor.
- **Continuidad de la Condición:** El hilo despertado retoma **fireTransition** con la posesión lógica del monitor. Como el *handoff* no libera **entry**, ningún otro worker puede modificar el marcado durante la transferencia; por eso el camino de *handoff* no vuelve a verificar la sensibilización. La verificación se realiza cuando el hilo adquiere **entry** mediante **ownership.acquire()**.

El monitor utiliza la **Policy** para decidir a quién despertar de forma selectiva. En lugar de un broadcast, construye la lista de transiciones sensibilizadas que tienen hilos esperando y delega en la política la selección de una de ellas. La selección se realiza en el método **selectWaiterOrRelease**: si existen candidatos elegibles, ejecuta **queues.signalOne(selectedTransition)** y transfiere la posesión mediante **ownership.handoff()**; de lo contrario, libera el mutex mediante **ownership.release()**.

3.3. Diagrama de Secuencia del Monitor

A continuación se presenta el flujo detallado de interacción entre el Worker, el Monitor y sus componentes auxiliares durante un ciclo de disparo.

[\[Clic aquí para abrir el Diagrama de Secuencia\]](#)

3.4. Diagrama de Clases

El diagrama de clases de este proyecto se encuentra a continuación.

[\[Clic aquí para abrir el Diagrama de Clases\]](#)

3.5. Flujo de Disparo de una Transición

El método principal es **fireTransition(int transition)**. Cuando un hilo intenta disparar una transición, sigue el siguiente flujo lógico:

1. **Toma del Mutex:** El hilo intenta adquirir el semáforo justo `entry`. Si está ocupado, se bloquea en ese semáforo. Si fue despertado mediante `handoff`, ya posee lógicamente el mutex y no necesita adquirir `entry` nuevamente.
2. **Verificación de Sensibilización:** Se ejecuta después de que el hilo adquiere `entry` mediante `ownership.acquire()`, ya sea al entrar por primera vez o al regresar de una espera temporal. Si el hilo retoma por un `handoff`, la posesión lógica ya está activa y esta verificación se omite.
3. **Bloqueo por Sensibilización (Wait):** Si la transición no está sensibilizada, el hilo incrementa el contador de espera, libera `entry` y se bloquea en el semáforo de la cola de esa transición. Al ser despertado mediante `signalOne`, recupera la posesión lógica con `wakeFromQueue()` y retoma el ciclo con `owned = true`, sin volver a adquirir `entry` ni repetir la verificación de sensibilización.
4. **Evaluación Temporal:** Si la transición está sensibilizada y el hilo posee el mutex, se consulta al objeto `TimeRestrictions`.
 - Si todavía no alcanzó α , el hilo libera el mutex mediante `ownership.release()`, espera mediante `awaitUntilEFT()` fuera de la sección crítica y luego vuelve al ciclo con `owned = false`; debe adquirir nuevamente `entry` antes de verificar la sensibilización.
 - Si alcanzó α , continúa con el disparo. La configuración vigente utiliza semántica débil con $\beta = \infty$, por lo que no existe expiración temporal.
5. **Disparo (Fire):** Se ejecuta el disparo en la RdP, actualizando el marcado y la sensibilización. Luego se actualiza el estado temporal, se comprueban los P-invariantes, se registra el disparo y se informa a la política.
6. **Señalización y Salida (Signal and Exit):** Antes de salir, el monitor ejecuta el método `selectWaiterOrRelease()`. Este busca las transiciones sensibilizadas que tienen hilos esperando. Si hay candidatos, la `Policy` selecciona una, `queues.signalOne(selectedTransition)` libera un permiso de su cola y se transfiere la posesión lógica mediante `handoff()`, sin invocar `entry.release()`. Si no hay candidatos, el monitor libera el mutex mediante `ownership.release()`.

3.6. Independencia de la Lógica

El método `fireTransition` realiza cada solicitud de disparo. Los workers son agentes “ciegos”: no inspeccionan el marcado para decidir una rama y solicitan únicamente las transiciones de su path fijo. Cuando el monitor debe despertar un waiter entre varias alternativas, consulta a `Policy`, que selecciona la transición de la próxima señal sin modificar el path del worker.

Para coordinar la finalización, la ejecución distingue el conteo de vueltas iniciadas del conteo de vueltas finalizadas. El worker cuyo path comienza con T_0 incrementa `startedInvariants` antes de iniciar una nueva vuelta y no permite superar el objetivo de 186 ciclos. El worker de T_{11} es el único que incrementa `completedInvariants` al completar su path. Ambos contadores son atómicos y sirven para coordinar la finalización, pero no establecen una correspondencia entre el T_0 que inició una vuelta y el token que finalmente consume T_{11} , porque los tokens de una Red de Petri ordinaria no tienen identidad.

4. TIEMPO

Teniendo el sistema funcionando, procedemos a implementar la semántica temporal; esto es, asignar tiempo a las transiciones temporales $\{T_1, T_4, T_5, T_8, T_9, T_{10}\}$.

Desarrollamos una semántica de tiempo débil (*weak-time*). Esta semántica establece que una transición temporizada puede ser disparada solo si el tiempo transcurrido desde que se habilitó se encuentra en el rango temporal $[\alpha, \beta]$. Luego, en la implementación, se optó por tomar $\beta = \infty$, definiendo el intervalo operativo como $[\alpha, \infty)$.

Al dejar el extremo abierto con ∞ , nos aseguramos de que α funcione simplemente como el tiempo mínimo que requiere cada tarea. Así, el sistema se vuelve mucho más flexible y capaz de absorber las demoras lógicas sin que la red se trabe o deje de funcionar.

4.1. Análisis Teórico

Definición de Tiempos Asignados

Transición	Tiempo [ms]	Descripción
T_1	100	Tiempo de espera en la cola de clientes
T_4	60	Atención de agente de reservas #1
T_5	60	Atención de agente de reservas #2
T_8	80	Gestión de cancelación de reservas
T_9	40	Procesamiento del pago de la reserva
T_{10}	40	Finalización del pago y preparación del retorno

Debido a la estructura de la Red de Petri, que presenta bifurcaciones para la atención y la resolución de la reserva, el Tiempo de Ciclo teórico (T_C) se calcula siguiendo los caminos lógicos independientes que un cliente puede atravesar. Dados los tiempos asignados, ambos caminos resultan equivalentes en tiempo, dando como resultado el tiempo de ciclo teórico para cualquier cliente individual:

$$T_C = \sum_i t_{min}(T_i) = 100 + 60 + 80 = 240 \quad [\text{ms}]$$

Este valor, bajo esta configuración temporal, también corresponde al tiempo por invariante.

A partir de esto, queda evidente que el cuello de botella del circuito es la transición T_1 , debido a que es la transición que individualmente requiere más tiempo para ser disparada. Al funcionar el sistema como una estructura segmentada e independiente, el sistema no podrá ser más rápido que la frecuencia de disparo de T_1 , limitando el *throughput* máximo a un cliente cada 100 ms en estado estacionario.

4.2. Análisis Práctico

El tiempo total teórico para procesar N ejecuciones de invariantes de transición se calculó de la siguiente manera:

$$T_{\text{teorico}} = T_C + (N - 1) \times t_{\text{bottleneck}}$$

Donde:

- T_C : Es el tiempo de ciclo para cualquier cliente individual.
- N : La cantidad de invariantes de tiempo.
- $t_{\text{bottleneck}}$: El tiempo de la transición mas lenta.

Para la realización de la tabla se tomaron 5 muestras de ejecuciones de cada escenario y se calculo el promedio entre ellas.

#	Escenario	Tiempo Teorico [ms]	Tiempo Practico [ms]	Desvio (%)
1	Tiempo balanceado	18.740	18.497	1,29
2	Etapas final 4 veces mas lenta	37.360	36.934	1,14
3	Agentes 4 veces mas lentos	22.620	22.466	0,68
4	Cola de clientes 2 veces mas rapida	14.990	14.549	2,94

Análisis de Resultados:

- **Escenario 1 (Base):** Funciona como punto de referencia. El tiempo total de ejecución está delimitado por la transición T_1 (100 ms), la cual actúa como el cuello de botella primario del *pipeline*, dosificando el flujo de tokens hacia el resto de las etapas independientes.
- **Escenario 2:** Al incrementar T_9 y T_{10} a 160 ms cada una, la sub-rama de confirmación pasa a requerir 320 ms en serie. Como un porcentaje de los clientes se bifurca obligatoriamente por este camino, la etapa de cierre se convierte en el nuevo cuello de botella absoluto del sistema. Esto genera una acumulación de hilos en las variables de condición previas, duplicando el tiempo total de ejecución de la red.
- **Escenario 3:** Al ralentizar simultáneamente ambos canales de atención en paralelo ($T_4 = T_5 = 240$ ms), la capacidad de procesamiento combinada de esa etapa se reduce a la mitad de su frecuencia individual, procesando idealmente un token cada 120 ms. Al ser este tiempo mayor al de T_1 , la etapa de agentes desplaza a T_1 como cuello de botella.
- **Escenario 4:** Al reducir el tiempo de T_1 a la mitad (50 ms), el cuello de botella original se acelera. Sin embargo, el cuello de botella del *pipeline* se desplaza inmediatamente a la siguiente etapa más lenta disponible en la red (en este caso, la gestión de cancelación $T_8 = 80$ ms o la suma de confirmación/pago), alcanzando el límite de saturación natural de la arquitectura concurrente actual.

5. POLÍTICAS

5.1. Objetivo y alcance de Policy

La clase `Policy` concentra el criterio de selección que se aplica durante el handoff del monitor. El proyecto define los modos `NONE`, `BALANCED` y `PRIORITIZED`. La política no modifica el marcado, no calcula la sensibilización de la Red de Petri y no decide los caminos de los workers. Su intervención sobre la ejecución se limita a elegir qué transición debe recibir la próxima señal cuando el monitor tiene candidatos elegibles. La clase también mantiene los contadores que se usan para informar los resultados.

La política no veta ni aplaza un disparo solicitado directamente por un worker. En cada iteración de `fireTransition`, el monitor comprueba si la transición del path fijo está sensibilizada y si puede dispararse según la restricción temporal. Si ambas condiciones se cumplen, la transición se dispara sin consultar a `Policy`.

Después de un disparo exitoso, el monitor busca transiciones sensibilizadas que tengan al menos un waiter en su cola. Si la lista no está vacía, llama a `Policy.choose()`, incluso cuando contiene un único candidato. La lista contiene candidatos para despertar, no decisiones directas de disparo. El resultado identifica la cola que debe recibir una señal y no reemplaza el path del worker.

5.2. Conflictos estructurales y oportunidades reales

Un conflicto estructural aparece cuando dos transiciones comparten plazas de entrada. En esta red se distinguen los siguientes grupos:

Grupo	Transiciones	Plazas compartidas	Significado
Agentes	T_2 y T_3	P_3	Atención por el agente superior o inferior.
Reservas	T_6 y T_7	P_9, P_{10}	Confirmación o cancelación de la reserva.

La existencia de un conflicto estructural no implica que la política deba elegir en cada disparo de una de esas transiciones. El monitor construye la lista de candidatos con la condición:

```
1 rdp.isSensitized(t) && queues.hasWaiters(t)
```

Hay una oportunidad real de resolver el conflicto de agentes cuando la lista contiene a T_2 y T_3 . Para el grupo de reservas, la condición equivalente es la presencia simultánea de T_6 y T_7 . Si sólo aparece una de las dos transiciones, no hay una elección proporcional para ese grupo y su residuo no se modifica.

La clase mantiene un residuo para cada grupo. Si ambos pares completos aparecen en la misma lista, `activeConflictIn()` examina primero el grupo de agentes y resuelve T_2/T_3 . En esa invocación no se actualiza el residuo de T_6/T_7 . Esta prioridad pertenece a la implementación actual y no debe confundirse con una independencia entre grupos: los residuos son independientes, pero la selección de un grupo por invocación no lo es.

5.3. Integración con el monitor

Tras un disparo exitoso, el monitor actualiza el marcado, registra el evento, llama a `Policy.recordFire()` y construye `wakingCandidates()`. Si la lista no está vacía, `Policy.choose()` selecciona una transición, `queues.signalOne()` despierta a un waiter y el monitor realiza el handoff. Si no hay candidatos, libera `entry`.

El handoff transfiere la posesión lógica sin liberar ni volver a adquirir `entry`. En los pares de conflicto, todas las transiciones son no temporizadas. Por eso, ningún otro worker modifica el marcado antes de que el waiter seleccionado retome la posesión y pueda disparar la transición en la reevaluación siguiente del ciclo.

La separación de responsabilidades es directa: `RdP` determina la sensibilización, `Queues` mantiene los waiters, `Policy` elige la transición y `Monitor` realiza la señalización y el handoff.

5.4. Selección proporcional mediante residuo

En los modos **BALANCED** y **PRIORITIZED**, la selección de un conflicto completo usa un acumulador entero. El valor acumulado representa la parte de la proporción objetivo que todavía no se convirtió en una elección discreta. Para cada conflicto del grupo, el algoritmo suma el porcentaje de la transición preferida. Si alcanza 100, selecciona esa transición y resta 100; de lo contrario, selecciona la alternativa y conserva el residuo.

El mecanismo es el siguiente:

```

1 accumulator += preferredPercentage;
2
3 if (accumulator >= 100) {
4     accumulator -= 100;
5     return preferred;
6 }
7
8 return alternative;
```

La secuencia no se escribe de forma manual. Surge de la acumulación y se corrige en cada decisión posterior. Si n conflictos reales usan un porcentaje preferido p , el número S_n de elecciones preferidas y el residuo A_n cumplen:

$$np = 100S_n + A_n, \quad 0 \leq A_n < 100. \quad (13)$$

Por lo tanto, $A_n/100$ es la fracción de una elección que queda pendiente. La cantidad observada puede apartarse momentáneamente de la ideal en menos de una elección, pero el residuo conserva esa diferencia para las decisiones siguientes.

Para el conflicto de agentes en modo **PRIORITIZED**, T_2 es la transición preferida, T_3 la alternativa y $p = 75$. Con residuo inicial igual a cero, se obtiene:

Conflicto	Acumulación	Elección	Residuo
1	$0 + 75 = 75$	T_3	75
2	$75 + 75 = 150$	T_2	50
3	$50 + 75 = 125$	T_2	25
4	$25 + 75 = 100$	T_2	0
5	$0 + 75 = 75$	T_3	75

En cuatro conflictos se seleccionan tres veces T_2 . La distribución del ejemplo es, entonces, $3/4 = 75\%$. La figura muestra la misma idea como una recta ideal y una escalera de decisiones enteras.

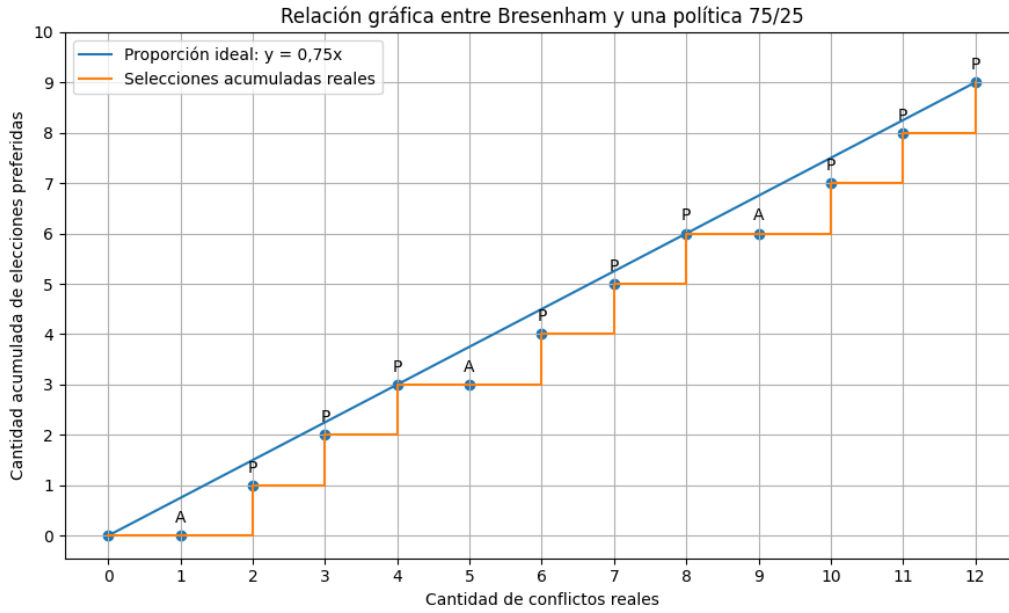


Figura 3: Acumulación de residuo para una proporción objetivo de 75/25.

5.5. Modos NONE, BALANCED y PRIORITIZED

La selección efectiva depende del modo y de que exista un par completo en la lista de candidatos:

Modo	Si hay un conflicto completo	Si no hay un par completo
NONE	Selecciona al azar entre todos los candidatos. No usa residuos.	Selecciona al azar entre todos los candidatos.
BALANCED	Usa 50 %/50 % para el grupo activo.	Selecciona al azar y no modifica ningún residuo.
PRIORITIZED	Usa $T_2/T_3 = 75\%/25\%$ o $T_6/T_7 = 80\%/20\%$, según el grupo activo.	Selecciona al azar y no modifica ningún residuo.

En BALANCED, el patrón alternado surge del acumulador de 50 %, no de una secuencia fija. En PRIORITIZED, las constantes privadas de Policy.java fijan 75 % para T_2 y 80 % para T_6 ; cambiarlas requiere modificar el código y recompilar. En cualquier modo, si no hay un par completo, choose() selecciona al azar entre los candidatos y no actualiza ningún residuo.

5.6. Resultados observados

Los resultados de las políticas sólo son comparables si se identifican el modo, la configuración y el origen de la evidencia. Para las corridas de comparación se mantuvieron la misma Red de Petri, los seis workers persistentes, el objetivo de 186 finalizaciones y las restricciones temporales base. El modo se fija mediante el valor de POLICY_MODE en Main.java; no se recibe como un argumento de ejecución.

Las tres capturas corresponden a la salida de Policy.printSummary() para esa configuración, con el modo indicado en cada subfigura. El resumen distingue los disparos globales registrados

por `recordFire()` de las decisiones proporcionales registradas por `recordConflictDecision()`. En `NONE` no se informa una resolución de Bresenham porque `choose()` utiliza `selectAny()`.

```
===== RESULTADOS =====
Policy mode: BALANCED

--- AGENTES (T2 vs T3) ---
Disparos totales: 186
  T2 (superior): 93 (50.0%)
  T3 (inferior): 93 (50.0%)

Conflictos: 186 (100.0%)
  Resolución Bresenham (objetivo: 50%):
    T2 elegida: 93 (50.0%)
    T3 elegida: 93 (50.0%)

--- RESERVAS (T6 vs T7) ---
Disparos totales: 186
  T6 (confirmadas): 93 (50.0%)
  T7 (canceladas): 93 (50.0%)

Conflictos: 186 (100.0%)
  Resolución Bresenham (objetivo: 50%):
    T6 elegida: 93 (50.0%)
    T7 elegida: 93 (50.0%)

=====
```

(a) Resumen de una corrida en modo `BALANCED`.

```
===== RESULTADOS =====
Policy mode: PRIORITIZED

--- AGENTES (T2 vs T3) ---
Disparos totales: 186
  T2 (superior): 139 (74.7%)
  T3 (inferior): 47 (25.3%)

Conflictos: 186 (100.0%)
  Resolución Bresenham (objetivo: 75%):
    T2 elegida: 139 (74.7%)
    T3 elegida: 47 (25.3%)

--- RESERVAS (T6 vs T7) ---
Disparos totales: 186
  T6 (confirmadas): 148 (79.6%)
  T7 (canceladas): 38 (20.4%)

Conflictos: 186 (100.0%)
  Resolución Bresenham (objetivo: 80%):
    T6 elegida: 148 (79.6%)
    T7 elegida: 38 (20.4%)

=====
```

(b) Resumen de una corrida en modo `PRIORITIZED`.

```
===== RESULTADOS =====
Policy mode: NONE

--- AGENTES (T2 vs T3) ---
Disparos totales: 186
  T2 (superior): 92 (49.5%)
  T3 (inferior): 94 (50.5%)

Conflictos: 0 (0.0%)

--- RESERVAS (T6 vs T7) ---
Disparos totales: 186
  T6 (confirmadas): 88 (47.3%)
  T7 (canceladas): 98 (52.7%)

Conflictos: 0 (0.0%)

=====
```

(c) Resumen de una corrida en modo `NONE`.

Figura 4: Resúmenes de las corridas en los tres modos de política.

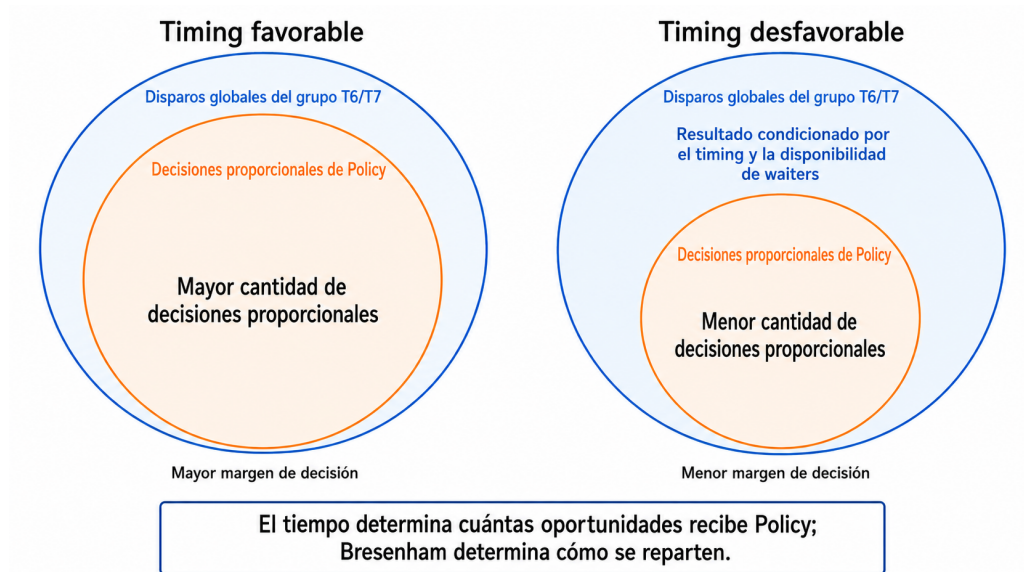


Figura 5: Influencia de la temporización sobre el margen de decisión de Policy.

La temporización no altera el cálculo proporcional de Bresenham, pero sí modifica cuántas veces ambos miembros del conflicto llegan juntos a Policy. Por eso, la política puede respetar su objetivo dentro de las decisiones donde interviene y no reproducirlo sobre todos los disparos globales.

5.7. Alcance y limitaciones

Para interpretar las métricas conviene separar cuatro cantidades:

- **Disparos globales:** todos los disparos registrados por `Policy.recordFire()`, incluidos los que no fueron precedidos por una elección entre dos ramas.
- **Handoffs:** invocaciones de `Policy.choose()` después de un disparo, siempre que exista al menos un candidato. Una lista de un solo elemento también produce una invocación, pero no una decisión de conflicto.
- **Oportunidades de conflicto:** invocaciones en las que aparecen completos T_2/T_3 o T_6/T_7 . Sólo estas invocaciones permiten aplicar Bresenham en los modos proporcionales.
- **Decisiones proporcionales y porcentaje observado:** elecciones registradas para el grupo que `activeConflictIn()` resolvió en esa invocación. Esta es la cantidad que debe compararse con los objetivos 75 %/25 %, 80 %/20 % o 50 %/50 %.

Los objetivos se comparan sobre decisiones proporcionales, no sobre disparos globales. La sensibilización, los tiempos y la disponibilidad de waiters determinan cuántas oportunidades existen. Si ambos grupos están completos en una misma invocación, la implementación resuelve primero el grupo de agentes; en modo `NONE`, los contadores específicos de decisiones proporcionales no se incrementan.

5.8. Conclusión

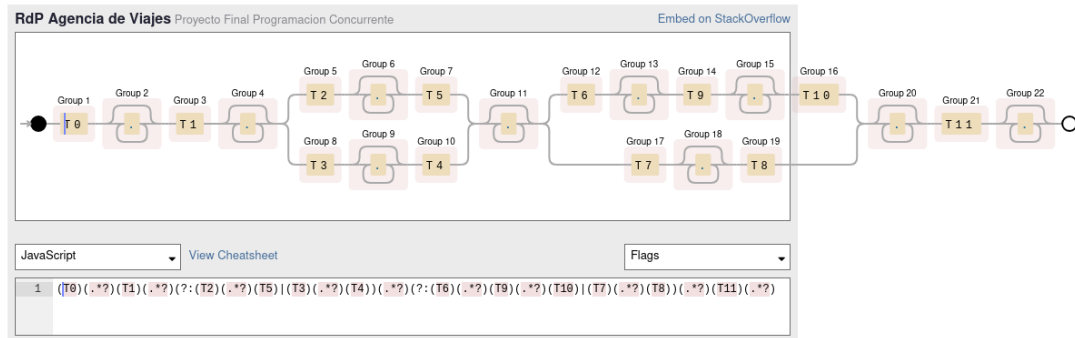
Policy selecciona una cola elegible durante el handoff, mientras el monitor conserva la exclusión mutua y actualiza la Red de Petri. Los objetivos proporcionales sólo describen los conflictos completos que llegan a `selectByPercentage()`, no todos los handoffs ni la distribución global de disparos.

6. ANÁLISIS DE T-INVARIANTES CON REGEX

Se utilizó la siguiente expresión regular para detectar las invariantes de transición (concatenadas como $T_0T_1T_2\dots$):

```
1 (T0)(.*)?(T1)(.*)?(?: (T2)(.*)?(T5) | (T3)(.*)?(T4)) (.*)?(?: (T6)(.*)?(T9)(.*)?(T10) | (T7)
  (.*)?(T8)) (.*)?(T11)(.*)?
```

El cual se ve expresado gráficamente en [Debuggex](#):



El patrón modela el orden lógico del invariante con ambas bifurcaciones luego de la sala de espera y de la aceptación/rechazo de reserva, respectivamente; el cual corresponde a las cuatro invariantes de transición mencionadas anteriormente.

El archivo `TInvariants.yaml` documenta los cuatro T-invariantes esperados. En la implementación actual, la expresión regular y el objetivo de 186 ejecuciones están definidos directamente en Python.

Aplicamos `(.*)?`, que representa cualquier secuencia intermedia de transiciones, que luego se guardan eliminando las transiciones que ya forman parte de un invariante. El uso de `'?'` permite que el cuantificador sea no codicioso (lazy), consumiendo lo mínimo necesario para hacer match con un invariante.

Los paréntesis representan grupos `'(...)'` que permiten reutilizar partes del match. El análisis repite el reemplazo hasta que no encuentra otra coincidencia; esto no implica que la secuencia quede vacía. El reemplazo se expresa en Python de la siguiente manera:

```
1 replacement = r"\g<2>\g<4>\g<6>\g<9>\g<11>\g<13>\g<15>\g<18>\g<20>\g<22>"
```

Los índices 2, 4, 6... representan solo a los interleavings. Con `'re.subn(..., count = 1)'` se reemplaza una ocurrencia por iteración, eliminando las transiciones que componen el invariante y preservando los interleavings restantes. El conteo se obtiene repitiendo este proceso hasta que no se encuentren más coincidencias.

Las bifurcaciones del modelo se representan con alternativas ‘or’ (A|B) agrupadas en ‘(?:...)’ para que se aplique sobre el tramo completo sin generar grupos capturantes extra, esto mantiene estable la numeración de grupos necesaria para el reemplazo.

6.1. Resultados Regex

Resultado de la ejecución del script de Python utilizando como entrada los logs:

```

Archivo de log cargado: logs/run.log
Cantidad de invariantes cargados: 4
Primeras transiciones: T0T1T2T5T6T0T9T1T2T5T10T0T11T1T3T4T6T0T9T1T2T5T10T

=== Reporte de invariantes ===
Invariantes detectados: 186
Invariantes esperados: 186
Resultado: OK

Detalle por tipo de invariante:
  T0-T1-T2-T5 / T6-T9-T10-T11: 112
  T0-T1-T2-T5 / T7-T8-T11: 28
  T0-T1-T3-T4 / T6-T9-T10-T11: 37
  T0-T1-T3-T4 / T7-T8-T11: 9

```

Figura 6: Salida de ejecución de REGEX sobre logs.

El analizador encuentra 186 invariantes completos. En la corrida mostrada no quedaron transiciones remanentes; si las hubiera, se imprimirían al final de la salida del analizador, no en el log original.

7. CONCLUSIONES

El trabajo implementó una simulación concurrente de la Red de Petri mediante seis workers persistentes con paths fijos. El monitor serializa el acceso al marcado y verifica la sensibilización y las restricciones temporales de cada transición solicitada. La política interviene durante el handoff para seleccionar qué hilo bloqueado despertar, sin modificar los paths ni elegir todos los disparos de la red.

La semántica temporal establece tiempos mínimos sin expiración y permitió comparar tiempos de ciclo y cuellos de botella en las configuraciones evaluadas. Las políticas balanceadas y priorizadas actúan únicamente cuando ambas ramas de un conflicto son elegibles. Por eso, sus objetivos describen esas decisiones y no necesariamente la distribución global de disparos. La temporización también puede modificar la cantidad de oportunidades en las que la política interviene.

En la corrida analizada, los chequeos de P-invariantes no registraron fallos y el analizador reconoció 186 secuencias compatibles con los cuatro T-invariantes, sin transiciones remanentes. Estos resultados respaldan la correspondencia entre el modelo y la implementación para la configuración evaluada.